MISSION ON MARS

Interactive Graphics 2025/26

Giacomo Bernardini (2070434)
Sapienza - Università di Roma



SAPIENZA
UNIVERSITÀ DI ROMA

Game Concept & Overview

The Premise

A catastrophic malfunction disables the **Artificial Sun** at a Martian extraction base. A lone rover must survive and restore power.



Dual-Phase Gameplay

▹ **Exploration Phase:**

Methodical **third-person** navigation to collect scattered energy cores using a robotic arm.

▹ **Combat Phase:**

Fast-paced **first-person** survival scenario against an algorithmic alien swarm, triggered upon objective completion.

Development Environment

- **WebGL & Three.js:**

WebGL provides hardware-accelerated **3D rendering in the browser**.

Three.js acts as the **core engine** simplifying the scene graph, lighting, and math utilities.

- **GLTFLoader:**

Utilized for **asynchronous loading of essential `.glb` models** (e.g., rover parts, energy batteries).

- **Physically Based Rendering (PBR):**

MeshStandardMaterial is used alongside Diffuse, Normal, and Roughness maps to physically anchor assets to the environment.

- **Decoupled UI:**

A comprehensive **HTML/CSS interface overlays the WebGL canvas**, ensuring crisp scaling and fluid DOM-to-Canvas interactions.

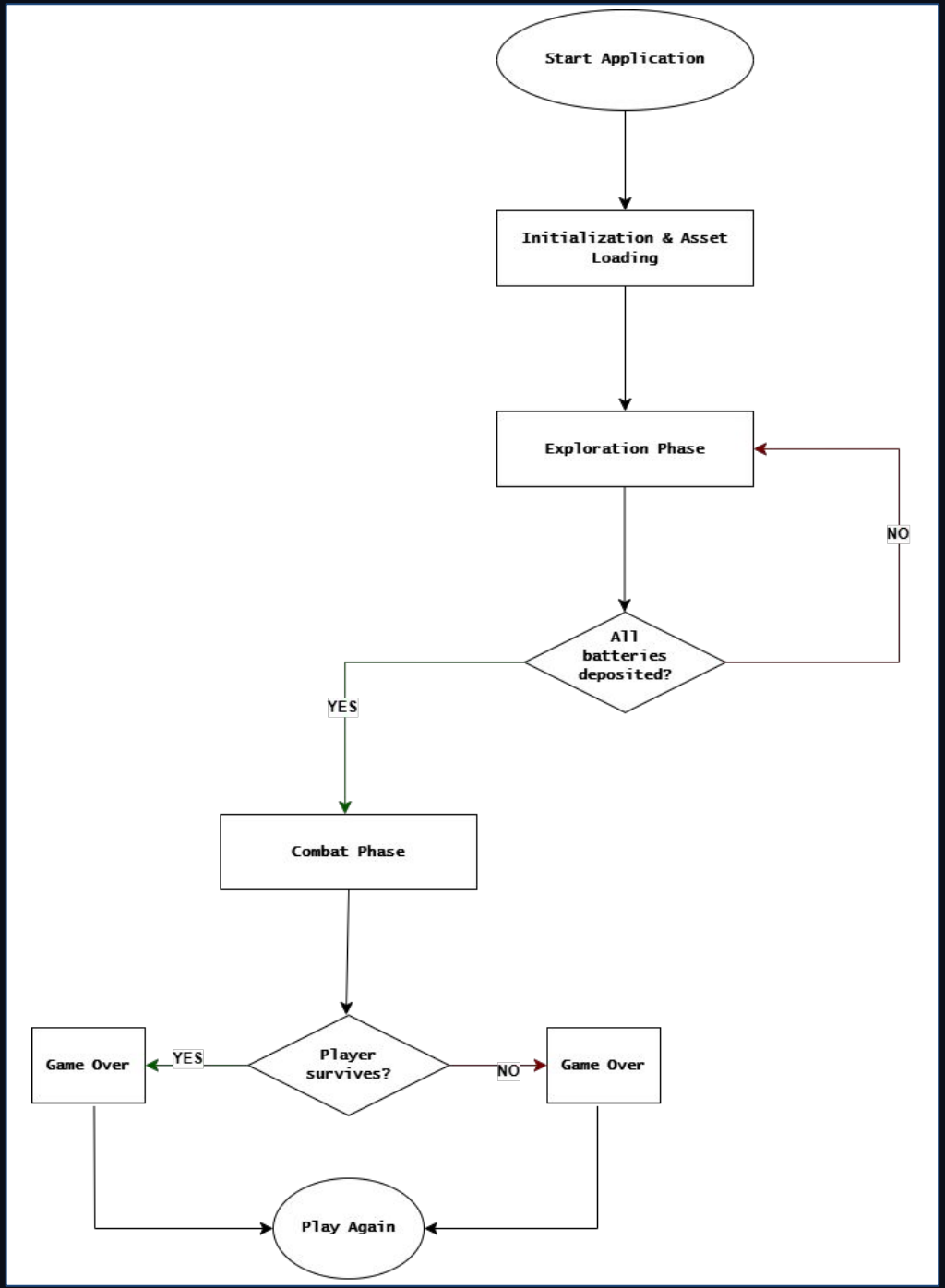
Software Architecture

1. Game State Management

The application **avoids logic conflicts** using a centralized state object containing boolean flags (e.g., gameStarted, missionWon). This dictates the active phase (Menu, Exploration, Combat) and **strictly limits which functions the main loop processes**.

2. The Main Loop (animate)

Driven by requestAnimationFrame, the central loop systematically **evaluates the active state to execute physics updates, algorithmic suspension, kinematics, and AI pathfinding** before instructing the renderer to draw the frame.



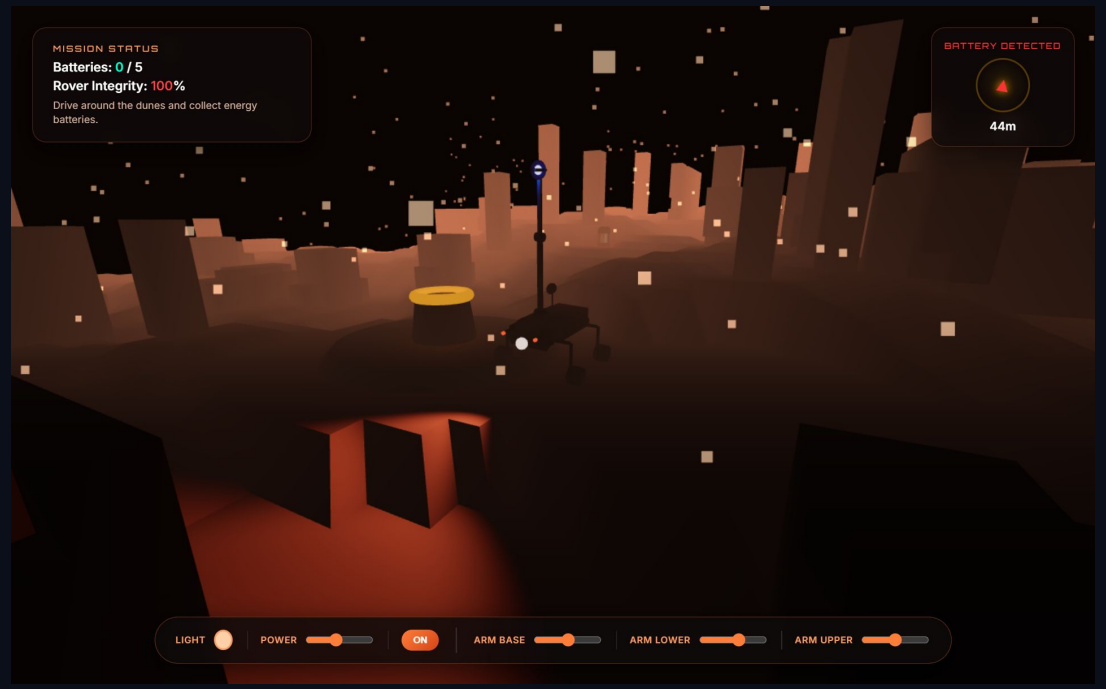
Procedural Environment

Algorithmic Martian Terrain

▹ Rather than importing static meshes, the terrain is a highly subdivided PlaneGeometry. **Vertex elevations are mathematically displaced** using composite **sine and cosine waves**.

▹ The selected difficulty level in the main menu directly dictates the **total number of energy batteries** required to spawn.

▹ Immersion is amplified by a custom **particle system**. Thousands of **dust particles** (using THREE.Points) dynamically update alongside Three.js **exponential fog**, creating a dense **sandstorm** with minimal performance overhead.



Dynamic Systems: Cameras & Lighting



Dual Cameras

Smooth transition from a third-person camera tethered to the rover via offset vectors, to a pointer-locked first-person turret view during combat.



Lighting Shift

Global illumination and fog colors **dynamically scale** and interpolate based on the ratio of collected batteries, simulating the **reignition of the Artificial Sun**.



Localized Lights

Interactive elements combine **high emissive intensity and localized PointLights**. This effectively highlights PBR textures and anchors entities without computing expensive dynamic shadows.

Trigonometric Navigation

- **Dynamic 3D Projection:**
The system mathematically **projects the 3D coordinates of active targets** (batteries, base, or extraction pad) **relative to the rover**.

- **Camera Synchronization:**
The calculation dynamically accounts for the **camera's current azimuthal angle** to ensure UI consistency regardless of the player's view.

- **Trigonometric Rotation:**
Utilizes the **Math.atan2(dx, dz) function to compute the exact rotation angle** for the 2D HTML compass arrow.

- **Real-Time Distance:**
Continuously calculates the **Euclidean distance** (Math.sqrt(dx*dx + dz*dz)) to provide accurate spatial feedback on the HUD.



Hierarchical Models & Forward Kinematics

Robotic Arm

- **Robotic Arm Hierarchy:**
The arm is constructed as a multi-node hierarchical chain (**armBase -> lowerPivot -> upperPivot -> magnetGroup**) attached to the rover's chassis.

- **Forward Kinematics:**
Rotations applied to parent nodes mathematically **cascade down to all child nodes**, preserving their local relative positions during real-time UI manipulation.

Aliens

- **Alien Swarm Assembly:**
The voxel-styled enemies are strictly generated programmatically as **THREE.Group hierarchies**. They feature interconnected nodes for the **body, head, core, leftShoulder, rightShoulder, leftWing, rightWing, leftLegs, rightLegs**, allowing isolated procedural animations for the relevant parts.

Procedural Physics & Animations

- **Raycaster Suspension:**
Downward-facing Raycaster objects continuously **read the procedural terrain's elevation**. The telescopic forks **scale dynamically**, and wheels adjust vertically using linear interpolation (lerp).

- **Terrain-Adaptive Velocity:**
The rover's movement speed is dynamically adjusted based on the chassis pitch (decelerating on steep dunes, accelerating downhill).

- **Gravity & Momentum:**
Dropped energy batteries are subjected to a custom physics simulation, **falling via gravity** and computing directional push forces to simulate **bouncy collisions on the hopper rim**.

- **Algorithmic Flight:**
Alien swarm movements rely entirely on **CPU-computed oscillating trigonometric functions** (Math.sin and Math.cos), driving the **hovering body**, **pulsating core**, and **wing flapping cycle** without pre-baked keyframes.

- **Native Procedural Death:**
Alien defeats trigger a **custom gravity-based backward fall and a voxel-style particle burst**, with trajectories and fading opacity computed strictly frame-by-frame.

Mathematical Swarm AI



Algorithmic Behaviors

Alien entities bypass heavy physics engines by utilizing radial collision detection. Their AI loop executes three core behaviors:

- **Obstacle Avoidance:** Computes **tangent vectors to slide around** procedural obstacles.
- **Separation:** Applies **repelling forces** to peers to **prevent overlapping meshes**.
- **Flanking & Braking:** **Encircles the player laterally** and **decelerates** upon reaching striking distance.

First-Person Combat Systems

- **Shooting Mechanics:**

Utilizes a Raycaster originating from the camera's center to simulate a **hit-scan laser cannon**.

- **Locational Hitboxes:**

Evaluates intersections against specific child meshes via userData. **Headshots neutralize targets**, while armor hits provide visual UI feedback.

- **Proximity Damage:**

Aliens **drain rover integrity strictly based on spatial proximity** rather than physical collisions.

- **Dynamic Difficulty:**

Scales **enemy spawn count**, **base speed**, and **laser cooldown** time mathematically based on menu selection.



Live Demonstration

Conclusion

Thank you for your attention.